# Verbalyn

## Real-Time Messaging Platform

| Nada Waleed 202301334 |
| --- |
| Aya Mostafa 202300348 |

## 1. Requirements Specification

### System Overview

Verbalyn is a real-time messaging platform that allows users to communicate instantly using a microservices architecture.

### Functional Requirements

#### Authentication

- Users can log in using Google OAuth2.
- The system generates JWT tokens for authentication.
- Users can log out.

#### User Management

- Users can view their profile (name, email, profile image).

#### Chat System

- Users can send messages.
- Users receive messages in real time.
- Users can view chat history.
- Users can edit their messages.
- Users can delete their messages.

#### Dashboard

- View active users.

- View number of messages sent.
- Monitor system activity.

### Notifications

- Users receive alerts for new messages.

## Non-Functional Requirements

### Scalability

- Horizontal scaling using containerized microservices

### Performance

- API response time under 300 ms under normal load

### Security

- OAuth2 authentication, JWT validation, HTTPS, rate limiting

### Reliability

- Retry mechanisms and circuit breakers to handle failures

### Observability

- Centralized logging, distributed tracing, and performance dashboards

## User Stories

### US-01

- A user logs in with Google to securely access the system.

### US-01

- A user sends a message to another user.

### US-03

- A user receives messages instantly without refreshing.

### US-04

- A user views past conversations.

### US-05

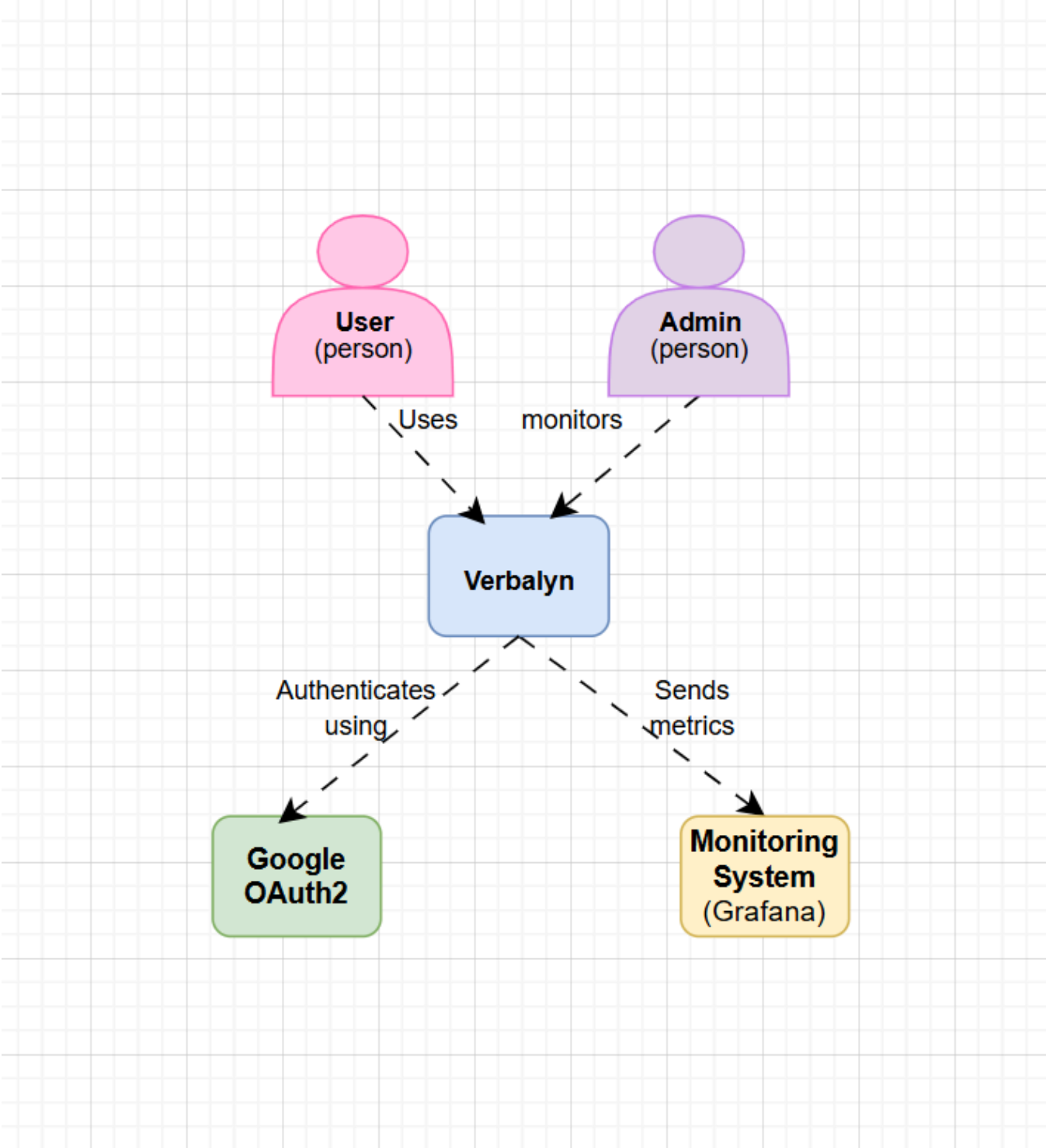
- An administrator monitors chat activity through the dashboard.

## 2. C4 Architecture Diagrams

All diagrams are written in PlantUML. Paste each block at plantuml.com to generate PNG/SVG images for the submission.

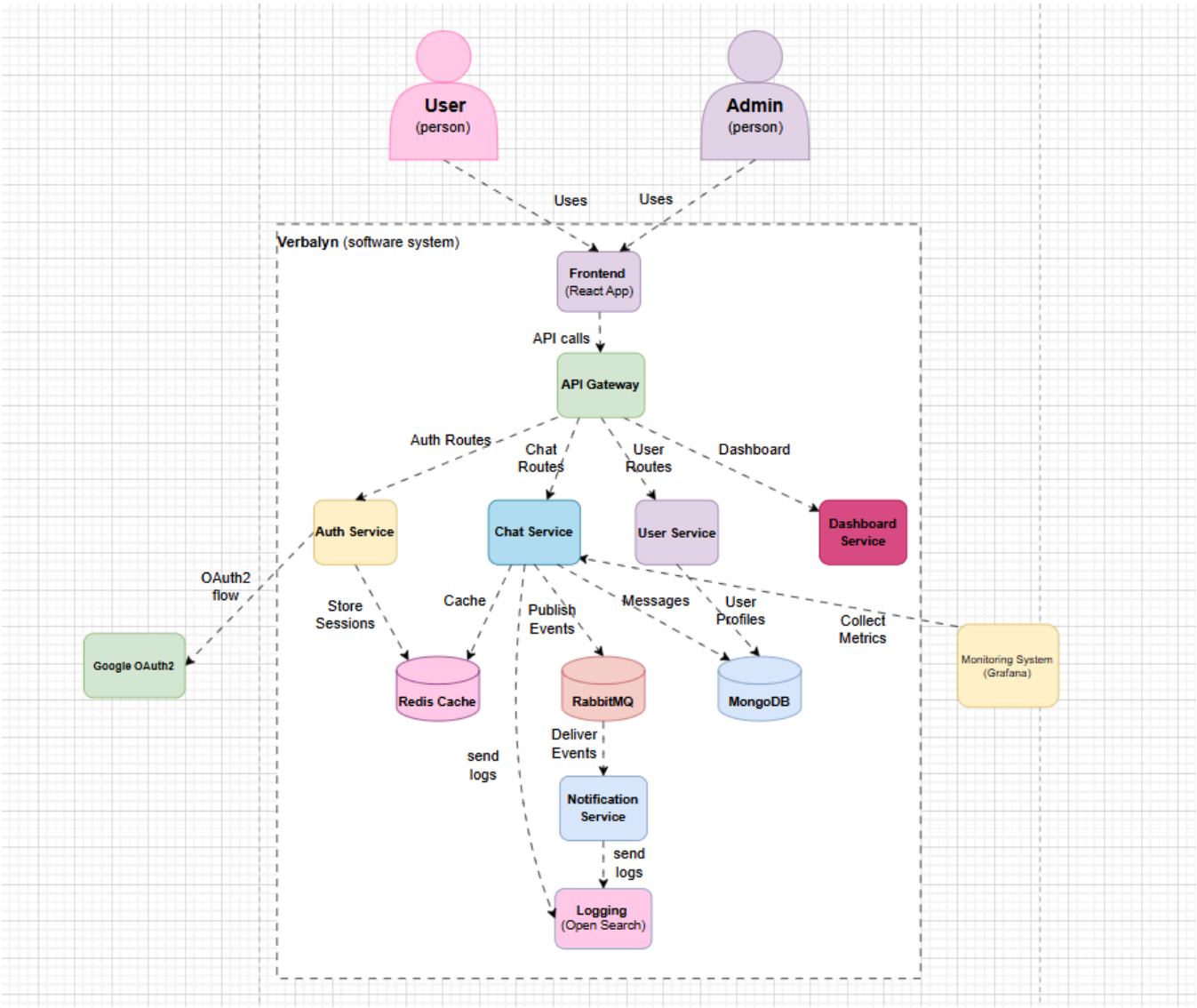
### 2.1 Context Diagram

Shows ChatSphere and how it connects to users and external systems.



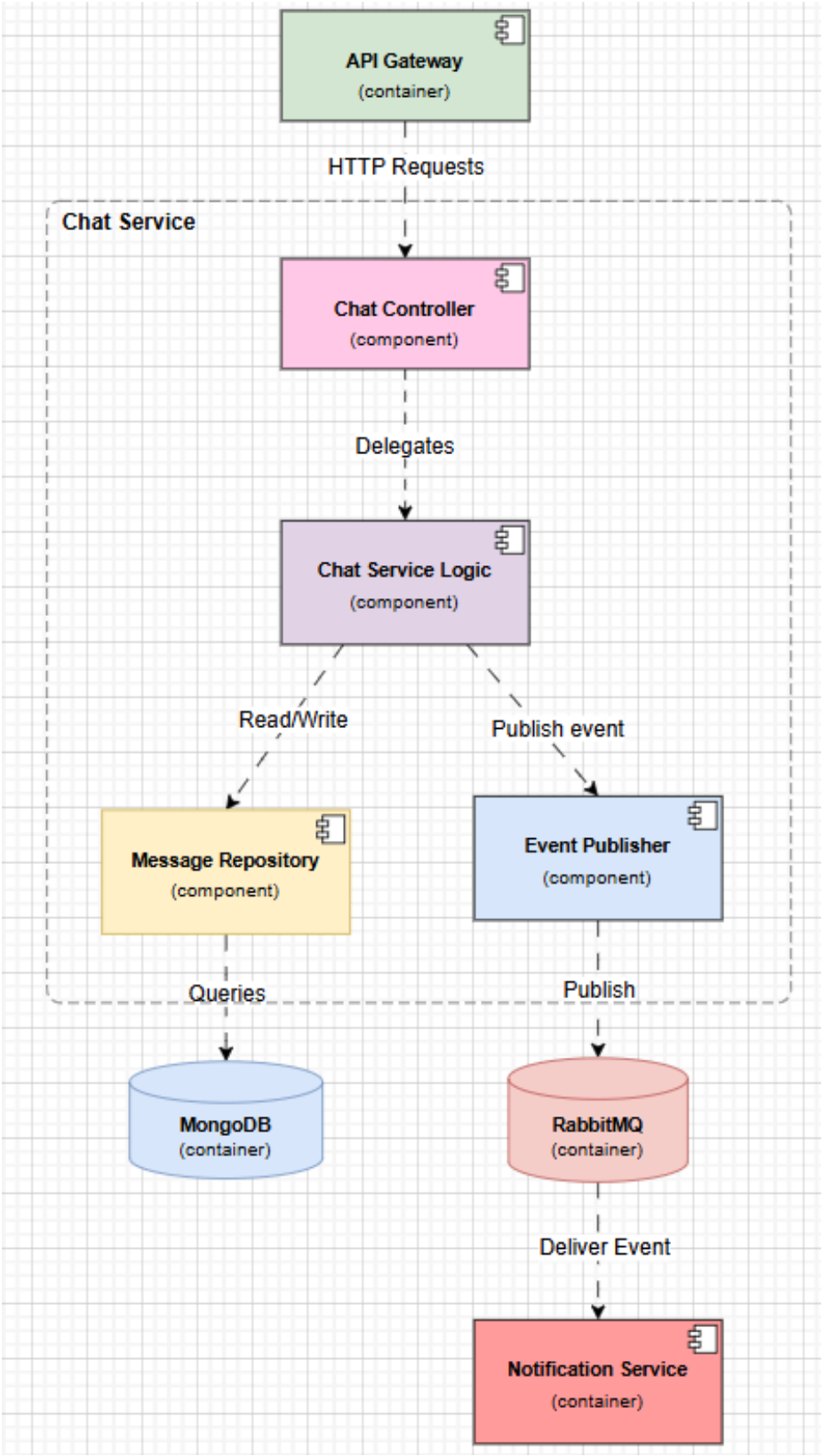
## 2.2 Container Diagram

Shows the services, databases, and infrastructure that make up ChatSphere.



## 2.3 Component Diagram — Chat Service

Shows the internal components of the Chat Service.



## 3. Architecture Decision Records

| ADR-001 — Microservices Architecture<br>Status: Accepted | |
|---|---|
| **Context** | The system must support independent scaling and deployment of features, and failures in one area should not affect others. |
| **Decision** | Use microservices instead of a monolithic architecture. |
| **Reasons** | Microservices allow each service to scale and be deployed independently, and a failure in one service does not affect the whole system. |
| **Benefits** | Independent deployment and improved scalability. |
| **Tradeoffs** | Increased complexity. |

| ADR-002 — MongoDB Database<br>Status: Accepted | |
|---|---|
| **Context** | Chat messages need flexible storage and fast R/W operations. |
| **Decision** | Use MongoDB as the primary database. |
| **Reasons** | MongoDB uses document-based storage which fits chat messages well. It also supports horizontal scaling. |
| **Benefits** | Flexible schema and  horizontal scaling. |
| **Tradeoffs** | Less strict schema enforcement. |

| ADR-003 — RabbitMQ Message Broker<br>Status: Accepted | |
|---|---|
| **Context** | Services need to communicate asynchronously so that one service failing does not block others. |
| **Decision** | Use RabbitMQ for asynchronous communication between services. |
| **Reasons** | RabbitMQ enables event-driven communication. For example, the Chat Service publishes an event and the Notification Service picks it up independently. |
| **Benefits** | Independent services and reliable message delivery. |
| **Tradeoffs** | Additional infrastructure component to operate. |

## ADR-004 — Redis Cache
**Status: Accepted**

| | |
|---|---|
| **Context** | Frequently accessed data such as sessions and recent messages should be served quickly without accessing the database every time. |
| **Decision** | Use Redis as a cache. |
| **Reasons** | Redis provides sub-millisecond key-value lookups, making it suitable for session storage and caching. |
| **Benefits** | Faster response times and reduced database load. |
| **Tradeoffs** | Cache consistency must be managed carefully. |

## ADR-005 — Google OAuth2 Authentication
**Status: Accepted**

| | |
|---|---|
| **Context** | The system must authenticate users securely without storing passwords. |
| **Decision** | Use Google OAuth2 for user login. |
| **Reasons** | Google OAuth2 is a trusted standard that doesn't store passwords. JWTs are made after login for next requests. |
| **Benefits** | Improved security, no password storage, easy user login. |
| **Tradeoffs** | Dependency on an external authentication provider. |

## ADR-006 — API Gateway
**Status: Accepted**

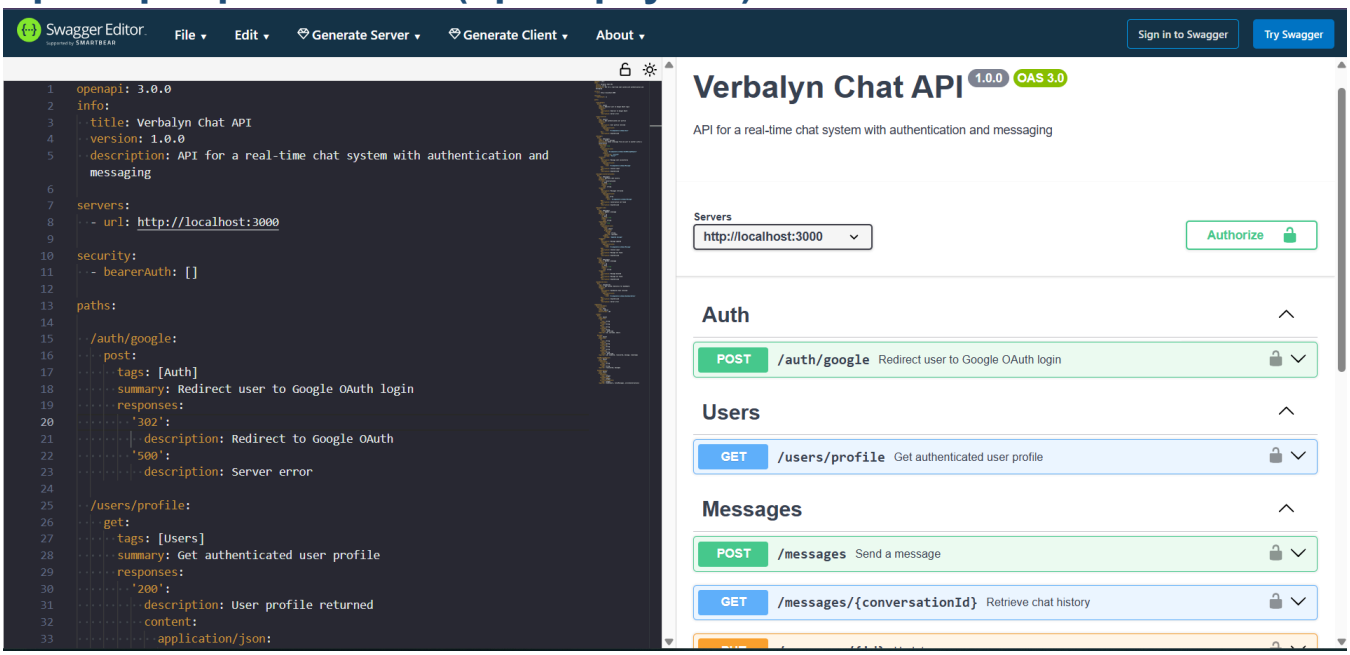
| | |
|---|---|
| **Context** | All client requests need a single entry point that handles routing, authentication, and rate limiting. |
| **Decision** | Use an API Gateway (Kong) in front of all microservices. |
| **Reasons** | Centralizing these concerns in the gateway means individual services do not need to implement them separately. |
| **Benefits** | Centralized routing, authentication enforcement, rate limiting. |
| **Tradeoffs** | The gateway is an additional component and must be kept highly available. |

# 4. Api Specification

All APIs are documented using OpenAPI 3.0. Key endpoints are listed below.

| Method | Endpoint | Description | Service |
|---|---|---|---|
| POST | /auth/google | Login with Google OAuth. Returns JWT token. | Auth Service |
| GET | /users/profile | Get the current user's profile. | User Service |
| POST | /messages | Send a message to another user. | Chat Service |
| GET | /messages/{conversationId} | Get chat history for a conversation. | Chat Service |
| PUT | /messages/{id} | Edit an existing message. | Chat Service |
| DELETE | /messages/{id} | Delete a message. | Chat Service |
| GET | /dashboard/stats | Get system statistics (active users, messages, etc.). | Dashboard Service |

## OpenApi Specification(openapi.yaml)



Swagger Editor — File ▾ Edit ▾ Generate Server ▾ Generate Client ▾ About ▾ — Sign in to Swagger — Try Swagger

```yaml
openapi: 3.0.0
info:
  title: Verbalyn Chat API
  version: 1.0.0
  description: API for a real-time chat system with authentication and
    messaging

servers:
  - url: http://localhost:3000

security:
  - bearerAuth: []

paths:

  /auth/google:
    post:
      tags: [Auth]
      summary: Redirect user to Google OAuth login
      responses:
        '302':
          description: Redirect to Google OAuth
        '500':
          description: Server error

  /users/profile:
    get:
      tags: [Users]
      summary: Get authenticated user profile
      responses:
        '200':
          description: User profile returned
          content:
            application/json:
```

## Messages

- POST /messages Send a message
- GET /messages/{conversationId} Retrieve chat history
- PUT /messages/{id} Update a message
- DELETE /messages/{id} Delete a message

## Dashboard

- GET /dashboard/stats Get system statistics for dashboard

### Schemas

```
User {
  id*            > [...]
  username*      > [...]
  email*         > [...]
  createdAt      > [...]
}
```

editor.swagger.io — Finish update

Swagger Editor — File ▾ Edit ▾ Generate Server ▾ Generate Client ▾ About ▾ — Sign in to Swagger — Try Swagger


```yaml
openapi: 3.0.0
info:
  title: Verbalyn Chat API
  version: 1.0.0
  description: API for a real-time chat system with authentication and
    messaging

servers:
  - url: http://localhost:3000

security:
  - bearerAuth: []

paths:

  /auth/google:
    post:
      tags: [Auth]
      summary: Redirect user to Google OAuth login
      responses:
        '302':
          description: Redirect to Google OAuth
        '500':
          description: Server error

  /users/profile:
    get:
      tags: [Users]
      summary: Get authenticated user profile
      responses:
        '200':
          description: User profile returned
          content:
            application/json:
```


```
User {
  id*            > [...]
  username*      > [...]
  email*         > [...]
  createdAt      > [...]
}

Message {
  id*            > [...]
  senderId*      > [...]
  receiverId*    > [...]
  message*       > [...]
  timestamp*     > [...]
}

SendMessageRequest {
  receiverId*    > [...]
  message*       > [...]
}

DashboardStats {
  totalUsers*            > [...]
  totalMessages*         > [...]
  activeConversations*   > [...]
}
```

# 5. Resilience Strategies

## Circuit Breakers

Circuit breakers stop sending requests to a service that keeps failing. This prevents one failure from spreading to other parts of the system.

Example: If the chat service fails a lot, the API Gateway stops forwarding requests to it for some time and returns an error instead.

## Retry Mechanisms

Failed requests are retried automatically before returning an error to the user.

- Maximum retries: 3
- Delay between retries increases

Example: Notification delivery retries up to 3 times if it failed.

## Message Queue Buffering

RabbitMQ holds messages in a queue when a service is temporarily unavailable. Once the service recovers, it processes the queued messages, which makes sure no data is lost.

Example: If the notification service is down, events stay in the RabbitMQ queue and are sent once its back up.

## Service Isolation

Each microservice runs in its own container. A failure in one service does not affect the others.

Example: If the dashboard service goes down, users can still send and receive messages.

# 6. Communication Architecture

## Synchronous Communication (REST)

Used when the client needs a fast response.

- Frontend to API Gateway to Service
- Protocol: HTTP

Example: A user sends a login request and waits for the JWT token response.

## Asynchronous Communication (RabbitMQ)

Used for background tasks where the client does not need to wait.

- Chat Service to RabbitMQ to Notification Service

Example: When a message is sent, the chat service publishes an event to RabbitMQ. The notification service picks it up and sends an alert without relying on the original request.

# 7. Monitoring and Observability

## Logging

Logs from all services are collected in OpenSearch.

- API requests and responses
- Service errors
- System events

## Metrics

Application and host metrics are collected using Prometheus.

### Application Metrics

- Request latency
- Error rate
- Throughput

### Host Metrics

- CPU usage
- Memory usage
- Disk utilization

## Dashboards

Grafana dashboards visualize system performance in real time.

- Messages per minute
- Active users
- Service health status